DBMS PROJECT PROPOSAL & DATABASE DESIGN

CampusHub

Smart Campus Facility, Booking & Inventory Management System

BCSE302P – Database Systems Lab

Relational Database Project | MySQL

TEAM MEMBERS

Mohammed Affan
Reg. No. 24BAI0014

Hardik Gupta
Reg. No. 24BCI0017

Arham Sadiq K
Reg. No. 24BAI0335

Introduction and Background

Many shared campus facilities

Classrooms, computer labs, seminar halls, sports facilities and other shared resources.

Manual or disconnected systems

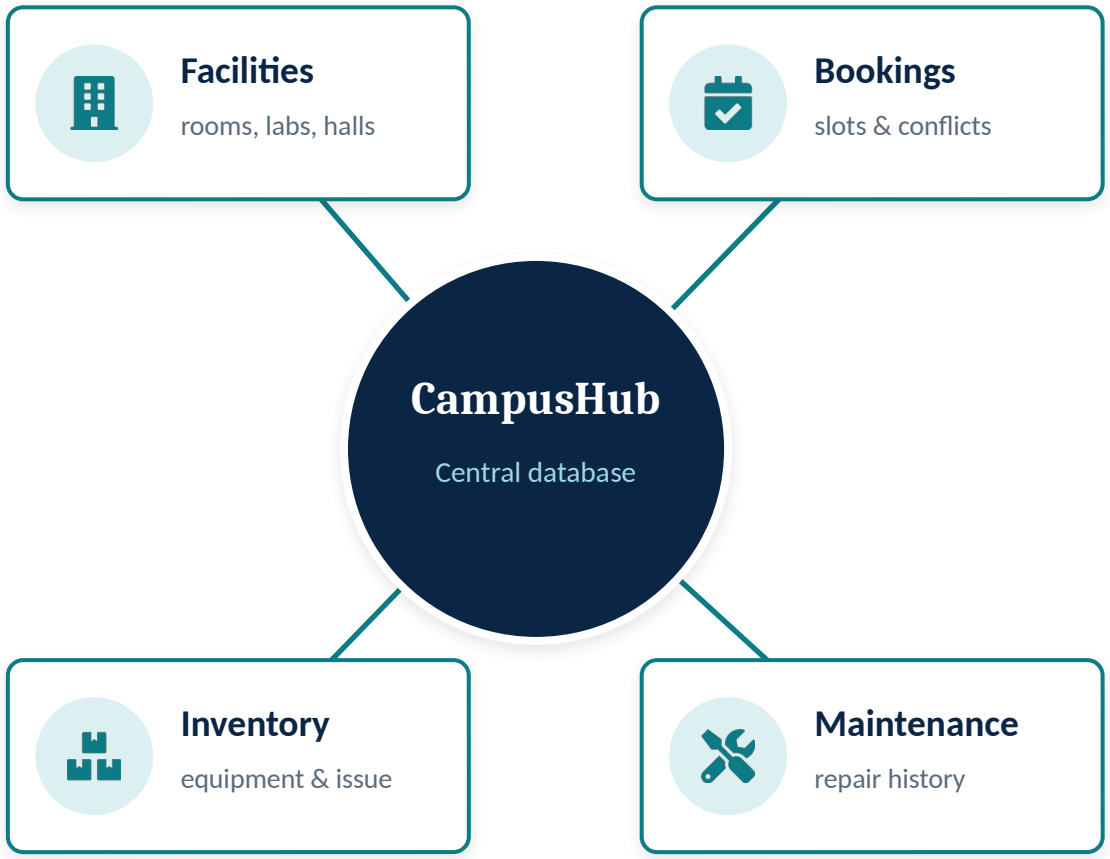
Cause problems in availability tracking, booking conflicts, equipment tracking and maintenance.

A centralized, database-driven solution

CampusHub keeps campus facility data in one relational database.

Connected information

Links facility management, booking, inventory and maintenance data.



Problem Statement

Real-time availability

Difficult to know which facility is free right now.

Booking conflicts

Double-booking of the same facility and time slot.

Equipment tracking

Hard to track equipment issued to students and faculty.

No central inventory

Equipment records are not kept in one place.

Maintenance history

Difficult to maintain a complete maintenance history.

Scattered records

Data is spread across disconnected sources.

Reports and statistics

Difficult to generate reports and usage statistics.

The need

A centralized relational database system for campus facilities, bookings, inventory and maintenance.

Objectives

1. Develop a centralized database for campus facilities.
2. Manage facility availability and bookings.
3. Maintain student and faculty information.
4. Track equipment inventory.
5. Record equipment issue and return transactions.
6. Maintain facility and equipment maintenance records.
7. Prevent booking conflicts using database constraints and transaction logic.
8. Generate useful reports using SQL queries.
9. Demonstrate DBMS concepts such as normalization, indexing, views, triggers and transactions.

Scope of the Project

INCLUDED

- Student management
- Faculty management
- Department management
- Facility management
- Facility booking
- Equipment inventory
- Equipment issue / return
- Maintenance management
- Booking history
- Usage reports
- Database-level validation

EXCLUDED

- Actual physical access-control hardware
- Real payment gateway
- IoT sensor integration
- Full mobile application
- Real-time building automation

CampusHub is primarily a DBMS / database project: the focus is data modelling, SQL, integrity and transactions.

Requirement Analysis

Data to be stored

- Student details
- Faculty details
- Department details
- Facility details
- Facility types
- Booking details
- Equipment details
- Equipment issue / return records
- Maintenance records

Users

- Students
- Faculty
- Facility administrators
- Department administrators

Operations

- Add / update / delete records
- Search facilities
- Check availability
- Create / cancel bookings
- Issue equipment
- Return equipment
- Record maintenance
- Generate reports

Information to retrieve

- Available facilities
- Current bookings
- Most frequently booked facilities
- Equipment availability
- Equipment currently issued
- Maintenance history
- Facility usage statistics

Database Selection and Justification



MySQL

Database type

Relational Database Management System (RDBMS)

Suitable for structured university management data.

Why MySQL is suitable

- ✓ Relational data model
- ✓ Transactions
- ✓ Primary and foreign keys
- ✓ Views
- ✓ Constraints
- ✓ Triggers
- ✓ SQL support
- ✓ Stored procedures
- ✓ Joins and aggregation
- ✓ Indexing

Proposal stage: MySQL is the proposed platform for the planned implementation. Nothing is claimed as implemented yet.

System Entities

Master data

DEPARTMENT
- department_id
- department_name

STUDENT
- student_id
- registration_no
- name
- email
- department_id

FACULTY
- faculty_id
- name
- email
- department_id

ADMIN
- admin_id
- name
- email
- role

FACILITY_TYPE
- facility_type_id
- type_name

Operational data

FACILITY
- facility_id
- facility_name
- building
- capacity
- status

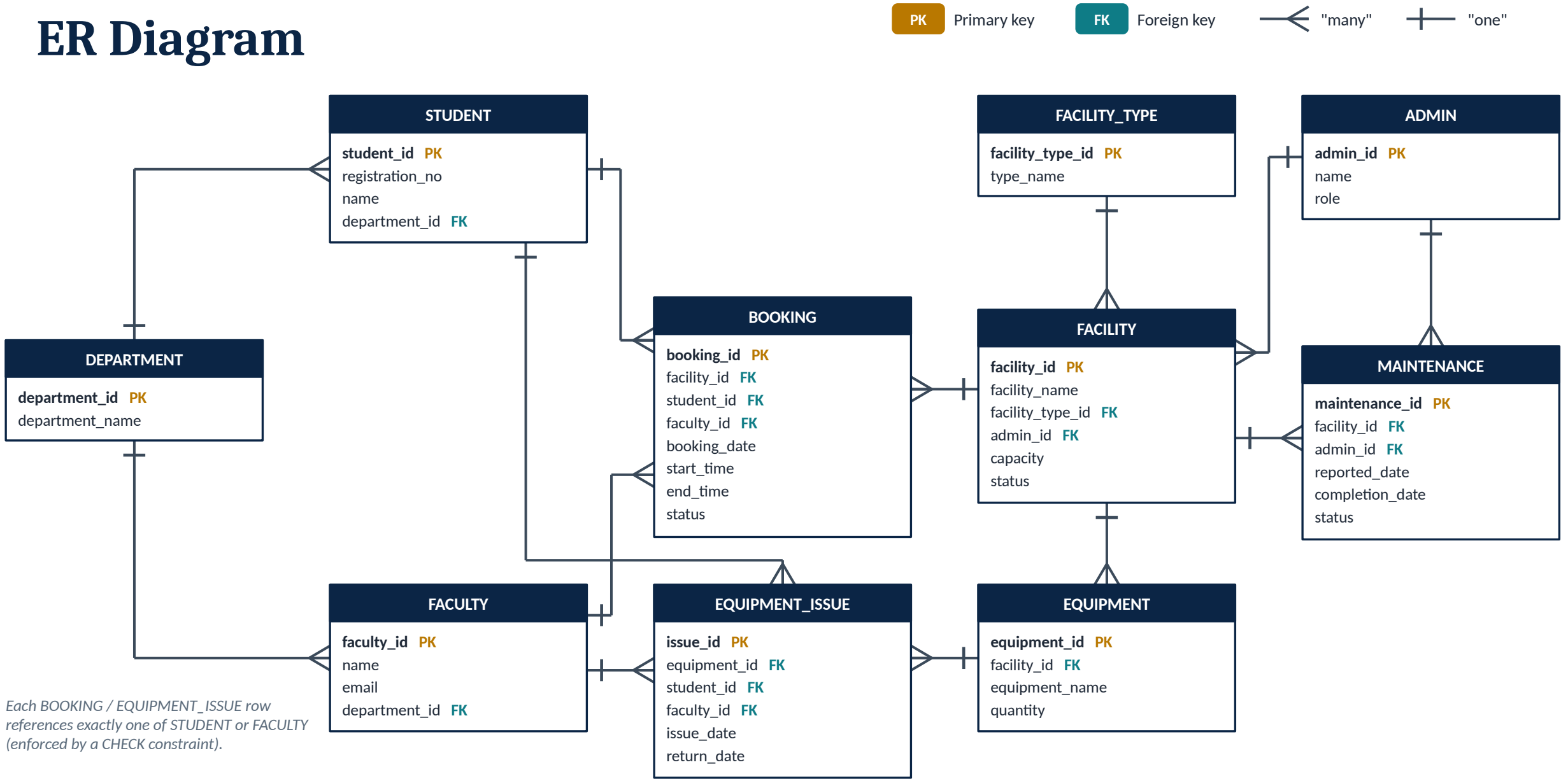
BOOKING
- booking_id
- facility_id
- booking_date
- start_time
- end_time

EQUIPMENT
- equipment_id
- equipment_name
- facility_id
- quantity
- availability_status

EQUIPMENT_ISSUE
- issue_id
- equipment_id
- issue_date
- return_date
- status

MAINTENANCE
- maintenance_id
- facility_id
- issue_description
- reported_date
- status

ER Diagram



Each BOOKING / EQUIPMENT_ISSUE row references exactly one of STUDENT or FACULTY (enforced by a CHECK constraint).

Relationships and Cardinality

Parent (“1” side)	Relationship	Child (“N” side)	Parent → Child	Child → Parent
DEPARTMENT	*has*	STUDENT	1 : N	N : 1
DEPARTMENT	*employs*	FACULTY	1 : N	N : 1
FACILITY_TYPE	*classifies*	FACILITY	1 : N	N : 1
FACILITY	*is reserved by*	BOOKING	1 : N	N : 1
STUDENT / FACULTY	*makes*	BOOKING	1 : N	N : 1
FACILITY	*houses*	EQUIPMENT	1 : N	N : 1
EQUIPMENT	*is issued via*	EQUIPMENT_ISSUE	1 : N	N : 1
STUDENT / FACULTY	*receives*	EQUIPMENT_ISSUE	1 : N	N : 1
FACILITY	*has*	MAINTENANCE	1 : N	N : 1
ADMIN	*manages*	FACILITY	1 : N	N : 1
ADMIN	*handles*	MAINTENANCE	1 : N	N : 1

Reading the cardinality

1 : N one parent row is linked to many child rows.

N : 1 each child row belongs to exactly one parent (via a foreign key).

Student / Faculty

A booking or equipment issue is made by either a student or a faculty member, so one of the two foreign keys is NULL. A CHECK constraint enforces this.

Relational Schema

DEPARTMENT(

department_id PK,
department_name UNIQUE

)

FACILITY_TYPE(

facility_type_id PK,
type_name

)

ADMIN(

admin_id PK,
name,
email,
role

)

STUDENT(

student_id PK,
registration_no UNIQUE,
name,
email,
department_id FK

)

FACULTY(

faculty_id PK,
name,
email,
department_id FK

)

FACILITY(

facility_id PK,
facility_name,
facility_type_id FK,
admin_id FK,
building,
capacity,
status

)

BOOKING(

booking_id PK,
facility_id FK,
student_id FK,
faculty_id FK,
booking_date,
start_time,
end_time,
status

)

EQUIPMENT(

equipment_id PK,
facility_id FK,
equipment_name,
quantity,
availability_status

)

EQUIPMENT_ISSUE(

issue_id PK,
equipment_id FK,
student_id FK,
faculty_id FK,
issue_date,
return_date,
status

)

MAINTENANCE(

maintenance_id PK,
facility_id FK,
admin_id FK,
issue_description,
reported_date,
completion_date,
status

)

PK primary key **FK** foreign key **UNIQUE** unique constraint

Keys and Constraints

Primary keys

A unique identifier for every entity, e.g. student_id, facility_id, booking_id.

Foreign keys

Maintain the relationships between tables, e.g. BOOKING.facility_id refers to FACILITY.facility_id.

NOT NULL

Mandatory columns must always hold a value.

```
facility_name
VARCHAR(100)
NOT NULL
```

UNIQUE

No two rows may share the same value.

```
UNIQUE
(registration_no)
```

CHECK

Rejects invalid data at the database level.

```
CHECK (capacity > 0)
CHECK (end_time >
  start_time)
```

DEFAULT

Value used when none is supplied.

```
status DEFAULT
  'CONFIRMED'
```

Referential integrity

A child row must point to an existing parent row.

```
FOREIGN KEY
(facility_id)
REFERENCES
FACILITY(facility_id)
```

Normalization up to 3NF

BEFORE: one unnormalized booking record (illustrative sample)

booking_id	student_name	reg_no	dept_name	facility_name	facility_type	building	capacity	equipment_used
101	Ravi	24XXX0101	Computer Science	Seminar Hall 1	Auditorium	Main Block	200	Projector, Mic
102	Meena	24XXX0102	Computer Science	Seminar Hall 1	Auditorium	Main Block	200	Mic

Repeated department and facility data, and a multi-valued equipment field, cause redundancy and update anomalies.

1NF — **Atomic values, no repeating groups**

equipment_used = "Projector, Mic" is split so each value is stored separately, in its own EQUIPMENT record.

2NF — **No partial dependencies**

Facility details depend on facility_id and student details on student_id, not on booking_id, so they move to FACILITY and STUDENT.

3NF — **No transitive dependencies**

facility_type depends on facility_id and dept_name on department_id, so they move to FACILITY_TYPE and DEPARTMENT.

AFTER

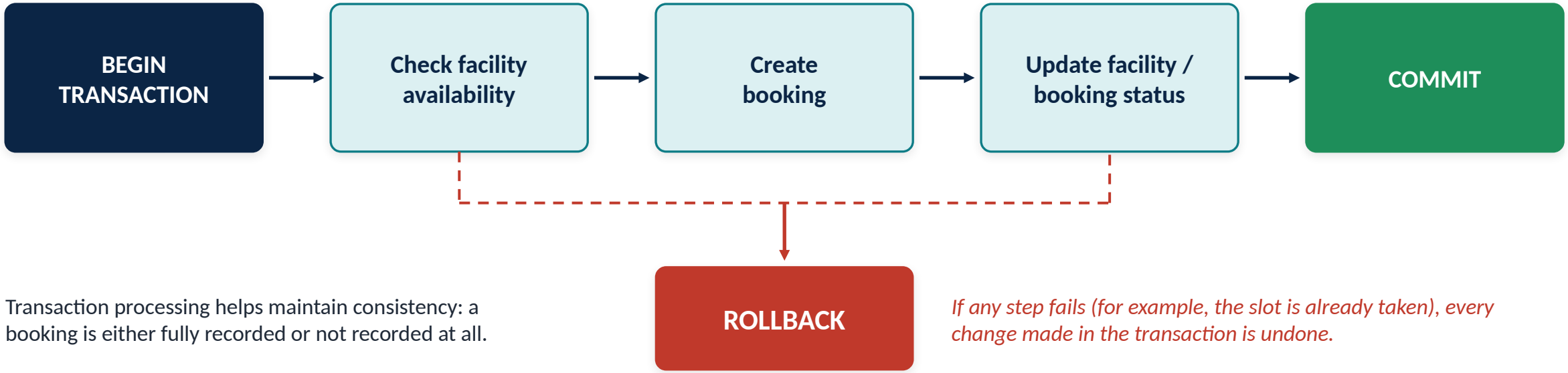
DEPARTMENT | STUDENT | FACILITY_TYPE | FACILITY | BOOKING | EQUIPMENT

Each fact is stored once, linked by keys. A capacity change is one update, not one per booking row.

No update, insert or delete anomalies

Transaction Management

Facility booking transaction



Transaction processing helps maintain consistency: a booking is either fully recorded or not recorded at all.

If any step fails (for example, the slot is already taken), every change made in the transaction is undone.

ACID properties

Atomicity

All steps succeed, or none do.

Consistency

Constraints hold before and after.

Isolation

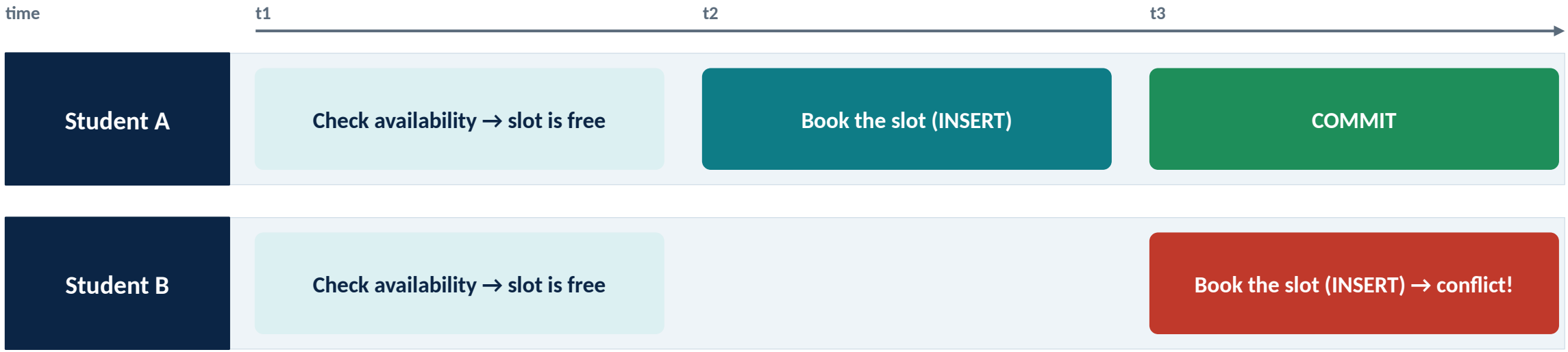
Concurrent bookings do not interfere.

Durability

Committed bookings persist.

Concurrency Control

Two students try to book the same facility and time slot at once



Without concurrency control, both checks see a free slot, so both bookings are accepted (double booking).

Isolation

Concurrent transactions should behave as if they ran one after another (e.g. a stricter isolation level such as SERIALIZABLE).

Locking

Lock the facility / slot rows during the availability check (e.g. SELECT … FOR UPDATE) so Student B waits until Student A commits.

Serializable schedule

The result equals a serial order (A, then B): B now sees the slot as taken and the booking is rejected.

SQL Operations and Queries

Planned queries

1. Find available facilities
2. Find all bookings for a facility
3. Find the most frequently booked facility
4. Find equipment currently issued
5. Find facilities with multiple maintenance records
6. Count bookings by department
7. Find facilities with capacity greater than a given value

Query 3 · Most frequently booked facilities (JOIN + GROUP BY)

```
SELECT f.facility_name, COUNT(*) AS total_bookings
FROM BOOKING b JOIN FACILITY f ON b.facility_id = f.facility_id
GROUP BY f.facility_id, f.facility_name
ORDER BY total_bookings DESC LIMIT 5;
```

Query 4 · Equipment currently issued (multi-table JOIN)

```
SELECT e.equipment_name, ei.issue_date, COALESCE(s.name, fa.name) AS issued_to
FROM EQUIPMENT_ISSUE ei JOIN EQUIPMENT e ON ei.equipment_id = e.equipment_id
LEFT JOIN STUDENT s ON ei.student_id = s.student_id
LEFT JOIN FACULTY fa ON ei.faculty_id = fa.faculty_id
WHERE ei.status = 'ISSUED';
```

Query 5 · Facilities with multiple maintenance records (HAVING)

```
SELECT f.facility_name, COUNT(*) AS maintenance_count
FROM MAINTENANCE m JOIN FACILITY f ON m.facility_id = f.facility_id
GROUP BY f.facility_id, f.facility_name
HAVING COUNT(*) > 1;
```

Representative examples of the proposed queries; the full set will be written during implementation.

Advanced DBMS Features

Triggers

PROPOSED

- Automatically update equipment and facility status after relevant operations
- e.g. equipment status changes when it is issued or returned

Views

PROPOSED

- Facility Usage Report
- Equipment Availability Report
- Maintenance History Report

Indexes

PROPOSED

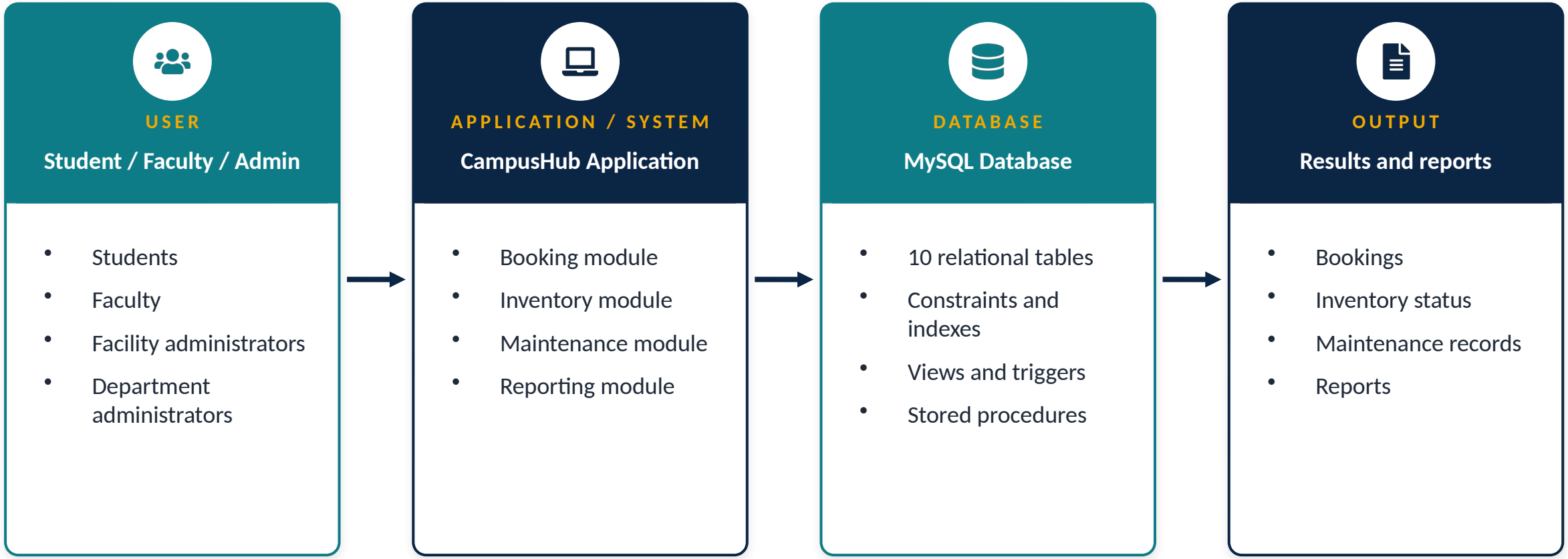
- Speed up searches and joins on commonly used columns:
- facility_id, booking_date, registration_no, equipment_id

Stored Procedures

PROPOSED

- Creating a booking
- Issuing equipment
- Recording maintenance

System Architecture / Workflow



Proposed workflow: users interact only through the application, and all data rules (constraints, transactions, triggers) are enforced in the MySQL layer.

EXPECTED OUTPUT

Expected Output / Reports

Facility availability report

Free facilities for a chosen date and time slot

Facility booking history

Past and current bookings for each facility

Most-used facilities

Facilities ranked by number of bookings

Equipment availability report

Equipment currently available in each facility

Issued equipment report

Equipment currently issued, and to whom

Maintenance history

Reported issues and completion status per facility

Department-wise facility usage

Bookings grouped by department

Monthly booking statistics

Number of bookings per month

Expected project outputs, to be produced through SQL queries and views. They are not yet implemented.

CONCLUSION

CampusHub provides a centralized relational database solution for campus facility, booking, inventory and maintenance management.



The project demonstrates

- ER modeling
- Relational schema
- Normalization
- SQL
- Constraints
- Transactions
- Concurrency
- Indexing
- Views
- Triggers
- Stored procedures

Thank You

CampusHub — Smart Campus Facility, Booking & Inventory Management System

TEAM

Mohammed Affan
24BAI0014

Hardik Gupta
24BCI0017

Arham Sadiq K
24BAI0335

BCSE302P – Database Systems Lab